Centro Universitário Presidente Antônio Carlos
UNIPAC – Barbacena

Sistema Especialista Inteligente para Mapeamento, Compatibilidade e Trilha de Carreira em TI

Relatório da disciplina de Inteligência Artificial

Aluno: Marco Antônio Milagres
Disciplina: Inteligência Artificial

Barbacena – MG
Outubro de 2026

Sumário

1 Introdução **2**

2 Fundamentação Teórica **2**
2.1 Sistemas especialistas baseados em regras 2
2.2 Incerteza e pontuação ponderada 3

3 Metodologia e Arquitetura **3**
3.1 Arquitetura geral 3
3.2 Base de conhecimento em JSON 3
3.3 Entrevista gradual 4
3.4 Motor de inferência 4
3.5 Regras e penalidades 5
3.6 Análise de gaps e roadmap 6

4 Implementação **7**
4.1 Estrutura do repositório 7
4.2 Trechos de código 7
4.3 Interface web em etapas 8

5 Resultados e Validação **8**
5.1 Personas 8
5.2 Casos-limite 8
5.3 Exemplo de diagnóstico 9

6 Conclusão **9**

1 Introdução

O mercado de tecnologia é amplo e repleto de ramificações, o que gera dúvidas em estudantes e profissionais sobre qual caminho seguir. Os sistemas especialistas clássicos, baseados apenas em regras SE-ENTÃO, funcionam bem quando o domínio é discreto, mas encontram limitações quando o mundo real é gradual: um usuário raramente "gosta" ou "não gosta" de uma área; ele gosta mais de algumas e menos de outras.

Este trabalho apresenta um **Consultor Avançado de Carreiras e Desenvolvimento Profissional** que combina uma base de conhecimento declarativa com um motor de inferência de **pontuação ponderada e tratamento de incerteza**. O sistema:

1. coleta respostas graduais (escala Likert de 1 a 5) e seleções de objetivo;
2. calcula uma matriz de compatibilidade percentual para dez trilhas de TI;
3. apresenta um ranking (Top 3) com percentuais de aderência;
4. identifica *skill gaps* em relação ao nível Júnior da trilha recomendada;
5. gera um roadmap personalizado (estudos, certificações e portfólio).

O desenvolvimento foi organizado em cinco fases: engenharia do conhecimento (JSON), motor de inferência, módulo de gaps e roadmap, interface web em etapas e validação com personas e casos-limite.

2 Fundamentação Teórica

2.1 Sistemas especialistas baseados em regras

Um sistema especialista é um programa que reproduz o raciocínio de um especialista em um domínio restrito [1]. A arquitetura tradicional separa:

- **base de conhecimento**: fatos e regras do domínio;
- **motor de inferência**: mecanismo que combina fatos e regras (encadeamento para frente ou para trás);
- **interface**: comunicação com o usuário e explicação das conclusões.

O desacoplamento entre base e motor é um requisito central: a base deve poder ser alterada (novos pesos, novas trilhas) sem modificar o código do raciocínio.

2.2 Incerteza e pontuação ponderada

Regras binárias não capturam respostas graduais. Uma alternativa amplamente utilizada é a **soma ponderada de atributos**, na qual cada resposta multiplica um peso que expressa a correlação entre aquele atributo e cada alternativa de conclusão. Esse modelo é equivalente, em espírito, aos *certainty factors* de MYCIN [2]: evidências favoráveis somam, evidências contrárias subtraem, e o resultado é normalizado.

Neste trabalho, o papel das "evidências contrárias" é desempenhado por **penalidades lógicas** aplicadas a perfis conflitantes (por exemplo, forte afinidade visual com pouca lógica deve reduzir o Back-end).

3 Metodologia e Arquitetura

3.1 Arquitetura geral

A Figura 1 resume o fluxo do sistema. A base de conhecimento fica em um arquivo JSON externo; o motor lê essa base, processa as respostas e devolve um diagnóstico estruturado; o módulo de recomendações cruza a trilha vencedora com os conhecimentos do usuário; a interface Streamlit orquestra o fluxo em etapas.

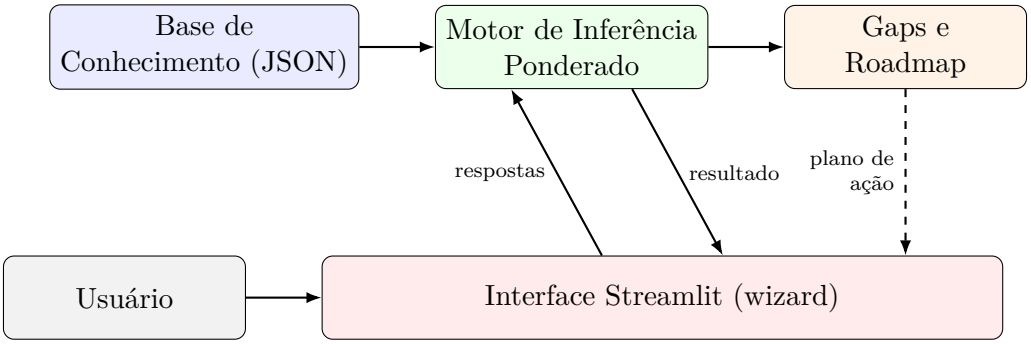


Figura 1: Arquitetura do sistema.

3.2 Base de conhecimento em JSON

O arquivo `knowledge/regras_carreiras.json` contém:

- **10 trilhas**, cada uma com nome, descrição, conhecimentos exigidos no nível Júnior e roadmap (estudos, certificações e projetos);
- **26 conhecimentos** catalogados e agrupados por categoria;
- **16 perguntas** (10 Likert e 4 de seleção) com pesos de correlação por trilha;
- **16 regras de bônus** e **7 penalidades lógicas**.

A seguir, um trecho ilustrativo de uma regra e de uma pergunta Likert:

Listing 1: Trecho da base de conhecimento (JSON).

```
{
  "id": "R07",
  "descricao": "Servidores + infraestrutura alta + arquitetura =
     Cloud",
  "condicoes": {
    "preferencia_diaria": "servidores",
    "infraestrutura": { "min": 4 },
    "aspecto_tecnico": "arquitetura"
  },
  "efeito": { "cloud": 12 }
}
```

3.3 Entrevista gradual

A entrevista é dividida em três etapas. A Tabela 1 resume as dimensões avaliadas.

Tabela 1: Dimensões da entrevista.

Etapa	Dimensão	Tipo
Perfil	Nível de experiência	seleção
Perfil	Conhecimentos já dominados	múltipla escolha
Afinidades	Lógica e algoritmos	Likert 1–5
Afinidades	Matemática e estatística	Likert 1–5
Afinidades	Design visual e interfaces	Likert 1–5
Afinidades	Servidores, redes e infraestrutura	Likert 1–5
Afinidades	Análise de dados	Likert 1–5
Afinidades	Segurança da informação	Likert 1–5
Afinidades	Automação, processos e qualidade	Likert 1–5
Afinidades	Gestão, liderança e negócio	Likert 1–5
Afinidades	Aplicativos móveis	Likert 1–5
Afinidades	Comunicação e pessoas	Likert 1–5
Objetivos	Preferência de atividade diária	seleção (9 opções)
Objetivos	Linguagem/ferramenta atual	seleção (4 opções)
Objetivos	Tipo de valor a entregar	seleção (5 opções)
Objetivos	Aspecto técnico mais atrativo	seleção (5 opções)

A Tabela 2 apresenta a matriz de pesos das afinidades (pontos por unidade da escala, de 1 a 5). As abreviações são: Back = Back-end, Front = Front-end/UI-UX, CD = Ciência de Dados, ED = Engenharia de Dados, Sec = Cibersegurança, DevOps, Cloud, Mob = Mobile, QA e PM = Product Management.

3.4 Motor de inferência

Seja L o conjunto de perguntas Likert, S o conjunto de perguntas de seleção e $r_q \in [1, 5]$ a resposta à pergunta q. Para cada pergunta de seleção, o_q é a opção

Tabela 2: Pesos das perguntas Likert (por ponto da escala).

Dimensão	Back	Front	CD	ED	Sec	DevOps	Cloud	Mob	QA	PM
Lógica	3	1	3	2	3	1	1	3	2	0
Matemática	2	0	5	3	1	1	1	0	1	1
Design	0	4	0	0	0	0	0	2	0	1
Infraestrutura	2	0	0	3	2	3	3	0	1	0
Dados	1	0	4	4	1	1	1	0	1	1
Segurança	1	0	0	1	5	2	2	0	1	0
Automação	1	0	1	2	1	3	2	0	4	1
Gestão	1	1	0	1	0	1	1	1	0	5
Mobile	1	2	0	0	0	0	0	5	0	1
Comunicação	1	2	1	0	1	1	1	1	1	4

escolhida e $p_{q,o,t}$ o peso dessa opção para a trilha t. O placar bruto da trilha t é:

$$S_t = \sum_{q \in L} r_q \, p_{q,t} + \sum_{q \in S} p_{q,o_q,t} + \sum_{k \in R_t} b_k - \sum_{k \in P_t} |b_k| \tag{1}$$

em que R_t e P_t são, respectivamente, as regras de bônus e as penalidades disparadas que afetam t, e b_k é o efeito da regra k. A pontuação máxima possível M_t representa o perfil perfeito para a trilha:

$$M_t = \sum_{q \in L} 5 \, p_{q,t} + \sum_{q \in S} \max_{o} p_{q,o,t} + \sum_{k \in R} \max(0, b_{k,t}) \tag{2}$$

A compatibilidade percentual é então:

$$C_t = \max\left(0, \ \min\left(100, \ \frac{100 \, S_t}{M_t}\right)\right) \tag{3}$$

O ranking ordena as trilhas por C_t decrescente. Empates são desfeitos pelo número de regras disparadas e, por último, pela ordem fixa do catálogo — garantindo que **sempre** exista uma recomendação, mesmo em perfis neutros ou com respostas vazias.

3.5 Regras e penalidades

As Tabelas 3 e 4 listam as regras de bônus e as penalidades lógicas. A notação "$\geq$" e "$\leq$" refere-se à resposta Likert.

Tabela 3: Regras de bônus.

ID	Condição (resumo)	Efeito
R01	Dados brutos, matemática ≥ 4 e Python	CD +12
R02	Dados brutos e análise de dados ≥ 4	CD +8, ED +2
R03	Pipelines e automação ≥ 4	ED +10
R04	Preferência por segurança	Sec +10
R05	Segurança e Linux/Redes	Sec +6

ID	Condição (resumo)	Efeito
R06	Servidores, automação ≥ 4 e aspecto automação	DevOps +10
R07	Servidores, infraestrutura ≥ 4 e aspecto arquitetura	Cloud +12
R08	Algoritmos, Python e entrega ao usuário	Back +10
R09	Layouts, JavaScript e aspecto criação	Front +10
R10	Mobile e JavaScript	Mob +10
R11	Qualidade e automação ≥ 3	QA +10
R12	Negócio e entrega de valor de negócio	PM +12
R13	Gestão ≥ 4 e comunicação ≥ 4	PM +8
R14	Design ≥ 4 e entrega ao usuário	Front +8
R15	Matemática ≥ 4 e dados ≥ 4	CD +8
R16	Infraestrutura ≥ 4 e Linux/Redes	DevOps +3, Cloud +3

Tabela 4: Penalidades lógicas.

ID	Condição (resumo)	Efeito
P01	Design ≥ 4 e lógica ≤ 2	Back -6, ED -4
P02	Lógica ≥ 4 e design ≤ 2	Front -6
P03	Matemática ≤ 2	CD -8
P04	Infraestrutura ≤ 2	DevOps -6, Cloud -6
P05	Automação ≤ 2	QA -5
P06	Segurança ≤ 1	Sec -5
P07	Gestão ≤ 2 e comunicação ≤ 2	PM -6

3.6 Análise de gaps e roadmap

Para a trilha vencedora, o módulo de recomendações compara o conjunto de conhecimentos marcados pelo usuário com a lista `nivel_junior_requerido` da trilha. O percentual de prontidão é:

$$\text{Pronto}(\%) = 100 \cdot \frac{|\text{dominados}|}{|\text{requeridos}|} \tag{4}$$

O roadmap prioriza os itens de estudo que preenchem as lacunas detectadas e, em seguida, apresenta os itens complementares, as certificações sugeridas e os projetos de portfólio.

4 Implementação

4.1 Estrutura do repositório

Listing 2: Estrutura de arquivos.

```
.
|-- app.py                          # Interface Streamlit (wizard)
|-- requirements.txt                # streamlit, pandas, plotly
|-- knowledge/
|   `-- regras_carreiras.json       # Base de conhecimento
|-- src/
|   |-- knowledge.py                # Loader do JSON
|   |-- motor.py                    # Motor ponderado
|   `-- recomendacoes.py            # Gaps e roadmap
|-- tests/
|   |-- personas.py                 # 10 personas
|   |-- edge_cases.py               # 11 casos-limite
|   `-- test_app.py                 # Teste de fumaca da interface
`-- docs/
    `-- relatorio-consultor.tex     # Este relatorio
```

4.2 Trechos de código

O núcleo do motor percorre as perguntas acumulando a contribuição ponderada e, depois, aplica as regras e penalidades:

Listing 3: Núcleo do cálculo de pontuação.

```python
placar = {t: 0.0 for t in kb.trilhas}

# 1) respostas ponderadas (Likert e selecoes)
for p in kb.perguntas:
    valor = respostas.get(p.id)
    if p.tipo == "likert" and valor is not None:
        for trilha, peso in p.pesos.items():
            placar[trilha] += valor * peso
    elif p.tipo == "selecao" and valor in p.pesos_por_opcao:
        for trilha, peso in p.pesos_por_opcao[valor].items():
            placar[trilha] += peso

# 2) regras de bonus e penalidades
for regra in kb.regras + kb.penalidades:
    if condicao_atendida(regra.condicoes, respostas):
        for trilha, valor in regra.efeito.items():
            placar[trilha] += valor

# 3) normalizacao 0-100%
compatibilidade = {
    t: max(0, min(100, round(100 * pts / maximo[t])))
    for t, pts in placar.items()
}
```

4.3 Interface web em etapas

A interface Streamlit foi organizada como um *wizard* de três passos, seguindo o padrão de navegação com estado de sessão:

1. **Passo 1**: perfil pessoal (experiência e conhecimentos atuais);
2. **Passo 2**: afinidades técnicas em dez controles deslizantes (1–5);
3. **Passo 3**: objetivos de carreira (quatro seleções);
4. **Resultado**: Top 3 em gráficos de barras e radar, tabela com o placar das dez trilhas, abas de *skill gaps* e plano de ação.

A execução local é feita com `streamlit run app.py`; a validação com personas e casos-limite, com `python -m tests.personas` e `python -m tests.edge_cases`.

5 Resultados e Validação

5.1 Personas

Foram construídas dez personas para cobrir as dez trilhas. A Tabela 5 mostra o resultado obtido (trilha recomendada e compatibilidade) e o Top 3.

Tabela 5: Validação com personas (10/10 corretas).

Persona	Perfil	Obtido	Top 3 (compatibilidade)
Ana	Analítico	CD (94%)	CD 94, ED 68, QA 51
Bruno	Infra/automação	DevOps (83%)	DevOps 83, Cloud 65, ED 58
Carla	Visual	Front (88%)	Front 88, Mob 60, PM 45
Diego	Negócios	PM (84%)	PM 84, Front 37, Mob 32
Eva	Arquitetura cloud	Cloud (83%)	Cloud 83, DevOps 68, ED 58
Fábio	Eng. de dados	ED (80%)	ED 80, DevOps 55, QA 53
Gabriela	Segurança	Sec (83%)	Sec 83, DevOps 54, Cloud 50
Henrique	Back-end	Back (73%)	Back 73, QA 51, ED 50
Isabela	Mobile	Mob (88%)	Mob 88, Front 73, PM 48
João	QA	QA (68%)	QA 68, Back 36, Mob 33

O sistema acertou a trilha esperada em todas as dez personas. Observa-se que os pares próximos são corretamente separados: Ciência de Dados versus Engenharia de Dados (matemática e "dados/insights" versus pipelines e infraestrutura), DevOps versus Cloud (automação versus arquitetura) e Front-end versus Mobile (JavaScript/layouts versus mobile explícito).

5.2 Casos-limite

Além das personas, foram implementados onze testes de robustez (Tabela 6), todos aprovados:

Tabela 6: Testes de robustez e casos-limite (11/11).

#	Caso testado
1	Perfil totalmente neutro (todas as respostas 3)
2	Respostas mínimas (todas 1)
3	Respostas máximas (todas 5)
4	Dicionário de respostas vazio
5	Respostas parciais (somente afinidades)
6	Perfil ambíguo (dados e desenvolvimento web)
7	Empate exato com desempate determinístico
8	Percentuais sempre entre 0 e 100
9	Gaps sem conhecimentos (roadmap prioriza lacunas)
10	Gaps com todos os conhecimentos (100%, sem pendências)
11	Conhecimento desconhecido no input ignorado com segurança

5.3 Exemplo de diagnóstico

Para a persona Bruno (perfil de infraestrutura), o sistema retornou: 1º DevOps com 83%, 2º Cloud com 65% e 3º Engenharia de Dados com 58%. As regras disparadas incluíram R06 ("servidores, automação ≥ 4 e aspecto automação") e R16 ("infraestrutura ≥ 4 e Linux/Redes"), além da contribuição das respostas Likert. O módulo de gaps apontou as lacunas de conhecimentos e o roadmap priorizou Docker, CI/CD e redes.

6 Conclusão

O trabalho atingiu os objetivos propostos: a base de conhecimento foi desacoplada do código em um arquivo JSON, o motor de inferência passou a lidar com respostas graduais e incerteza por meio de soma ponderada com penalidades lógicas, e a recomendação deixou de ser única para se tornar um ranking percentual. O sistema identifica *skill gaps* e gera um plano de ação individualizado, e a interface em etapas torna a experiência mais próxima de uma consultoria real.

Os resultados de validação (10/10 personas e 11/11 casos-limite, incluindo respostas vazias) confirmam a robustez do motor: ele sempre produz uma recomendação coerente e justificável. Como trabalhos futuros, pode-se ampliar o catálogo de trilhas, calibrar os pesos com especialistas e incorporar feedback do usuário para ajuste contínuo do modelo.

Referências

[1] RICH, E. *Artificial Intelligence*. New York: McGraw-Hill, 1983.

[2] SHORTLIFFE, E. H. *Computer-Based Medical Consultations: MYCIN*. New York: Elsevier, 1976.

[3] NILP0INTER. *Experta: Expert Systems for Python*. Repositório GitHub. Disponível em: https://github.com/nilp0inter/experta. Acesso em: out. 2026.

[4] STREAMLIT. *Streamlit Documentation*. Disponível em: https://docs.streamlit.io. Acesso em: out. 2026.

[5] PANDAS. *pandas Documentation*. Disponível em: https://pandas.pydata.org/docs/. Acesso em: out. 2026.

[6] PLOTLY. *Plotly Python Open Source Graphing Library*. Disponível em: https://plotly.com/python/. Acesso em: out. 2026.